

Notebook Details — Task_4.ipynb

Purpose

A **Twitter Sentiment Analysis** pipeline — from raw tweet data to trained ML classifiers with evaluation and visualization.

Dataset

- **File used:** twitter_training.csv
- **Columns:** Tweet ID, Entity, Sentiment, Tweet content
- **Sentiment classes:** Positive, Negative, Neutral, Irrelevant

Libraries Used

Category	Libraries
Data handling	pandas, numpy
NLP	nltk, TextBlob, wordcloud
ML	sklearn (LogisticRegression, MultinomialNB, LinearSVC, TfidfVectorizer)
Visualization	matplotlib, seaborn
Model saving	joblib
Regex/text	re, string

Notebook Structure (Cell by Cell)

1. Imports & Setup

Downloads NLTK resources (stopwords, punkt, wordnet) and imports all required libraries.

2. Data Loading & Renaming

Loads `twitter_training.csv`, renames the default columns to human-readable names (Tweet ID, Entity, Sentiment, Tweet content), and saves the renamed version as `twitter_training_renamed.csv`.

3. Class Distribution Inspection

Prints value counts for the Sentiment column showing how many Positive, Negative, and Neutral tweets exist.

4. Text Preprocessing Pipeline (Basic)

- Converts tweet content to string
- Removes digits using regex
- Removes special characters (keeps only letters and spaces)

5. Extractive Summarization Demo

A manual demonstration of NLTK tokenization and stopwords removal on a sample sentence — shows original text, tokenized words, and filtered words after removing stopwords.

6. Full Preprocessing Function (v1 — NLTK-based)

Defines `preprocess_tweet()` using:

- NLTK stopwords removal
- WordNetLemmatizer (lemmatization)
- PorterStemmer (stemming)

Applies it to the dataset to create a `Tweet_clean` column.

7. Train/Test Split

Uses `train_test_split` with `test_size=0.3`, `random_state=42`, and `stratify=y` to preserve class distribution.

8. TF-IDF Vectorization

Configures `TfidfVectorizer` with:

- English stopwords
- `max_df=0.95`, `min_df=2`

- Unigrams + bigrams (`ngram_range=(1,2)`)
- Sublinear TF scaling

9. Logistic Regression Classifier

Trains `LogisticRegression(max_iter=1000)` and prints a classification report.

10. Multi-model Comparison (v2 — cleaner preprocessing)

Refines preprocessing (no stemmer/lemmatizer, custom stopword set), reduces test size to 0.2, then trains and compares:

- **Naive Bayes** (`MultinomialNB`)
- **Linear SVM** (`LinearSVC` with balanced class weights)

11. Full Evaluation Pipeline (v3 — final version)

The most complete version — includes:

- URL removal in preprocessing
- Loops over both models
- `evaluate_model()` function that computes **accuracy, per-class precision, recall, F1**
- Plots **confusion matrix heatmaps** for each model
- Saves models + vectorizer to `sentiment_models.joblib` using `joblib`

12. Sentiment Distribution Bar Chart

Bar plot (seaborn viridis palette) of tweet count per sentiment class.

13. WordCloud Visualization

Generates one WordCloud per sentiment category (Positive, Negative, Neutral, Irrelevant) from the cleaned tweet text.

Output Files

- `twitter_training_renamed.csv` — renamed dataset
- `sentiment_models.joblib` — saved TF-IDF vectorizer + trained models

Summary Report

Notebook: Task_4.ipynb — Twitter Sentiment Analysis

Objective: Build and evaluate ML models that classify tweets into sentiment categories (Positive, Negative, Neutral, Irrelevant) using NLP preprocessing and TF-IDF feature extraction.

Data: Twitter training dataset with four columns — Tweet ID, Entity (brand/game name), Sentiment label, and Tweet content.

Approach:

The notebook follows a standard NLP classification pipeline. Raw tweets are cleaned by lowercasing, removing URLs, special characters, and stopwords. Two preprocessing strategies were explored — a heavier NLTK-based approach (stemming + lemmatization) and a lighter regex-based one. The final version uses the lighter approach for speed. Features are extracted using TF-IDF with unigrams and bigrams.

Models Trained: Logistic Regression (exploratory), Naive Bayes (MultinomialNB), and Linear SVM (LinearSVC with balanced class weights). The final pipeline evaluates Naive Bayes and Linear SVM with full per-class metrics and confusion matrix visualizations.

Evaluation: Each model is assessed on accuracy, precision, recall, and F1-score per class. Confusion matrices are plotted as heatmaps. Trained models are persisted to disk with `joblib`.

Visualizations: Sentiment class distribution bar chart and per-sentiment WordClouds are generated to understand the data and model behavior intuitively.

Key Takeaway: The notebook iteratively improves from a basic single-model setup to a robust, reusable evaluation framework with model saving — making it portfolio-ready for NLP/ML roles.